

Machine Learning of Families of Fluid Operators

Final Presentation

Princeton Liu, Joseph Wang

August 12, 2026

Intro to PDE Operator Learning

Given a PDE:

$$f\left(u, \frac{\partial u}{\partial x}, \frac{\partial u}{\partial t}, \frac{\partial^2 u}{\partial x^2}, \frac{\partial^2 u}{\partial x \partial t}, \frac{\partial^2 u}{\partial t^2}, \dots\right) = 0$$

And initial condition:

$$u_0 := u(\cdot, 0)$$

We want to find the evolution of u_0 with respect to time.

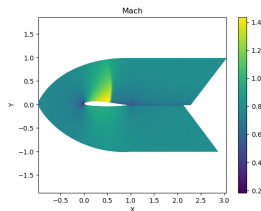
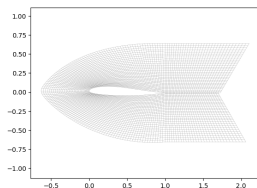
Sometimes we want to find a steady-state flow $u(\cdot)$, and sometimes we want to find, for some fixed T

$$u(\cdot, T) = ?$$

Operator Learning

In general, we want to approximate a nonlinear PDE operator by some neural network $G : U \rightarrow V$, with the following formulation:

$G[u_0](x) \sim u(x)$, In general, we cannot do this approximation for all points x , but rather we learn a discretization of u_0 and map it to a discretization of u .



Multi-Operator Learning

Given a single-operator network $G_i : U \rightarrow V$, multi-operator learning aims to leverage a family of single operator networks to learn decoupled parameters α that the single-operator networks do not see.

We can do this by passing in α into a separate operator network, and using a linear combination given by the following sum:

$$G[\alpha, u](x) = \sum_{i \in I} \underbrace{l_i[\alpha](x)}_{\text{partition}} \underbrace{G_i[u](x)}_{\text{single network}} \quad (1)$$

Lifted Multi-Operator Network

We can modify the Multi-Operator network by doing the recombination of the decoupled parameters at a higher latent space, specifically the computation step of the FNO Branches. Such a network looks like:

$$G[\alpha, u](x) = Q \left(\sigma \left(\sum_{i \in I} \underbrace{\tilde{l}_i[\alpha] \cdot \tilde{G}_i[u]}_{\text{latent space product}} (x) \right) \right) \quad (2)$$

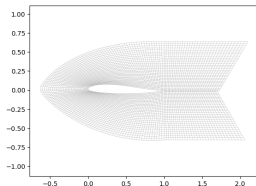
This allows the network more flexibility during the recombination step, instead of doing it over R , which improves accuracy.

Airfoil Problem

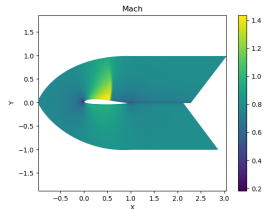
In the airfoil problem, the parameters u and α are the following:

u is the geometry of the airfoil, denoted in data by the (x, y) coordinates of all grid points.

α consists of three parameters: the airflow speed M , the angle of attack θ , and the air heat capacity γ .



$$\alpha = [1.4, 0.8, 0] \rightarrow$$



FNO Architecture

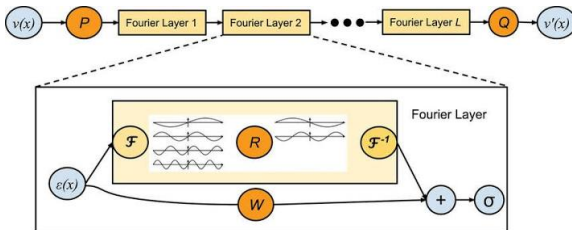


Figure: FNO Architecture

FNO Architecture

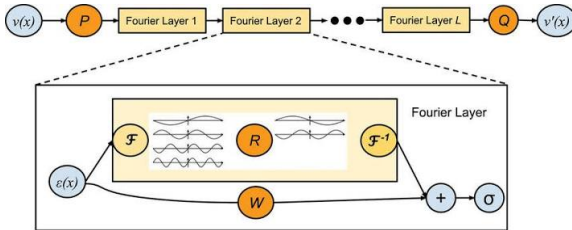


Figure: FNO Architecture

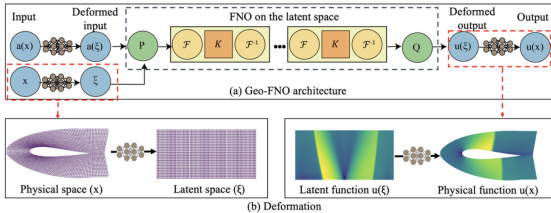


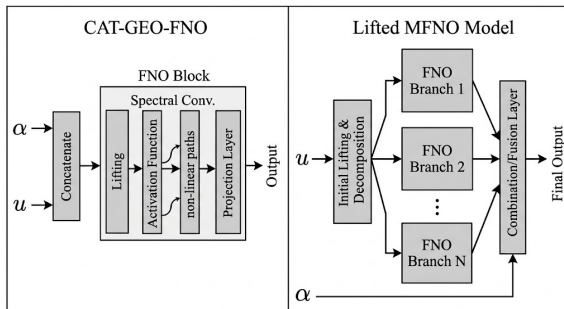
Figure: GeoFNO Architecture

Airfoil Problem: Models

We proposed and trained two model architectures as neural surrogate solvers for the airfoil problem.

The first model, CAT-GEO-FNO, concatenates α and u together during the forward pass.

The second model, Lifted MFNO, follows the model architecture described in the previous slide while using FNO models as branches.



Airfoil Problem: L2 Error

	9 Datasets	16 Datasets	25 Datasets
GEO-FNO			11.8%
CAT-GEO-FNO	3.99%	3.34%	2.99%
Lifted MFNO	4%	3.39%	3.01%

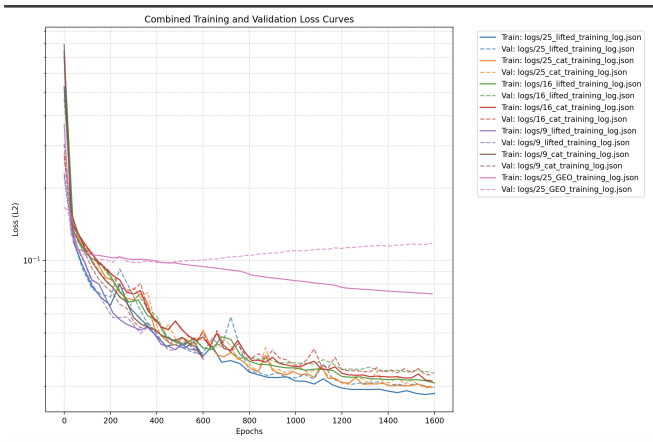


Figure: LossCurve

Airfoil Problem: Inference Speed

Both proposed models were able to infer about 10,000x faster than a traditional numerical solver.

This shows that at the cost of about 1-2 hours of training per model, we were able to generate accurate (3% – 5%) neural surrogate solvers for the airfoil problem.

For downstream tasks, we expect that the different models will distinguish themselves for the many different classes of problems.

Airfoil Problem: Conclusion

We explored different model architectures for creating neural surrogate solvers for a PDE steady-state problem.

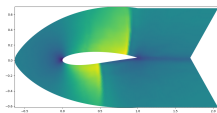
The PDE steady-state problem was finding the steady-state airflow around an airfoil given the geometry of the airfoil, and the ambient conditions (angle of attack, etc.)

To do so, we designed two different model architectures, CAT-GEO-FNO and Lifted MFNO. Both models were able to reach 3% – 5% error on substantially large datasets, and infer at a much higher speed than a traditional numerical solver.

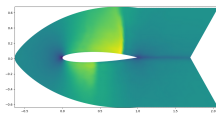
For future work after the program, we expect there to be more optimizations in the multi-operator network.

Airfoil Problem: Inverse GEO-Optimization

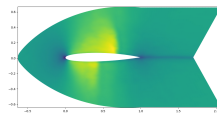
We used our trained model to do an inverse optimization problem around the geometry of the airfoil. The model reaches steady state geometry very quickly, and reverts to the standard naca-airfoil($\theta = 0$) in under 60 epochs at 0.001 learning rate.



(a) Epoch 0: 5.4 L/D Ratio



(b) Epoch 30: 13 L/D Ratio



(c) Epoch 60: 28.1 L/D Ratio

Figure: Progression of L/D Ratio over 60 epochs.

Navier-Stokes Problem

We study 2D incompressible flow on a periodic square $[0, 2\pi]^2$, written in vorticity form:

$$\frac{\partial \omega}{\partial t} + (\mathbf{v} \cdot \nabla) \omega = \frac{1}{Re} \Delta \omega \quad (3)$$

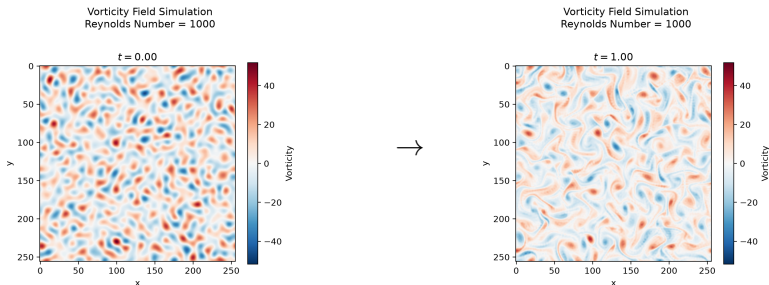
where $\omega = \nabla \times \mathbf{v}$ is the vorticity and Re is the Reynolds number.

Low Re flows smooth out quickly (viscosity dominates), high Re flows stay turbulent (advection dominates).

Navier-Stokes Problem

We want to learn a family of operators indexed using Re

$$G[Re] : \omega(\cdot, t) \mapsto \omega(\cdot, t + \Delta t), \quad \Delta t = 0.1$$



Data Generation: Pseudo-Spectral Solver

We generate ground-truth data using a pseudo-spectral solver.

Cost: roughly 20 seconds per trajectory on CPU (~ 5000 solver steps to reach $T = 1$).

We sample the operator family hierarchically:

- ▶ 100 Reynolds numbers, log-uniform in $[100, 3000]$
- ▶ 50 random initial conditions per Reynolds number
- ▶ 11 vorticity snapshots per trajectory at $t = 0, 0.1, \dots, 1$

Consecutive snapshots give one-step training pairs $(\omega_t, \omega_{t+0.1})$, so 5000 trajectories yield 50,000 pairs.

We hold out along both axes of the family:

- ▶ Unseen IC: 5 initial conditions held out at each training Re
- ▶ Unseen Re : 10 Reynolds numbers held out entirely

Is a Family Even Necessary?

FNOs which are trained on a single Reynolds number fail to generalize for a wide range of inputs.

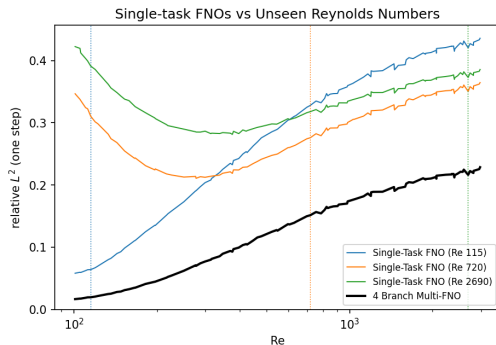


Figure: Single- Re FNOs vs. 4 Branch Multi-FNO across Re

Navier-Stokes Problem: Models

We test the same family of architectures as the airfoil problem, adapted to time-stepping on a uniform grid.

CAT-FNO (concatenate input naively for a single FNO):

$$G[\alpha, u](v) \approx \text{FNO}[\alpha, u](v) \quad (4)$$

Multi-FNO (linearly combine outputs from multiple FNOs):

$$G[\alpha, u](x) = \sum_{i \in I} \underbrace{l_i[\alpha](x)}_{\text{partition}} \underbrace{G_i[u](x)}_{\text{single network}} \quad (5)$$

Lifted Multi-FNO (Multi-FNO but combination of outputs happens before final layer):

$$G[\alpha, u](x) = Q \left(\sigma \left(\sum_{i \in I} \underbrace{\tilde{l}_i[\alpha] \cdot \tilde{G}_i[u]}_{\text{latent space product}}(x) \right) \right) \quad (6)$$

Navier-Stokes Problem: Models

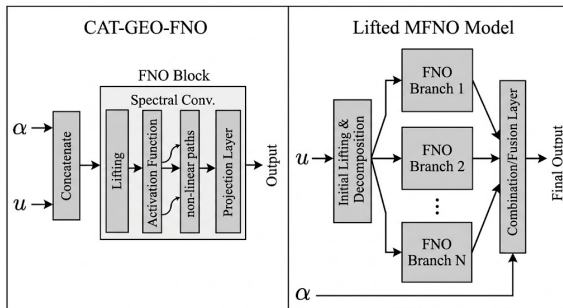


Figure: Model Architectures

L2 Error

	Params	Unseen ICs	Unseen Re
CAT-FNO (width 32)	4.7M	13.4%	17.1%
CAT-FNO (width 64)	18.9M	11.2%	14.4%
Multi-FNO (4 branches)	18.9M	13.1%	16.8%
Multi-FNO (32 modes)	33.6M	8.1%	10.4%
Lifted MFNO	18.9M	7.1%	9.0%

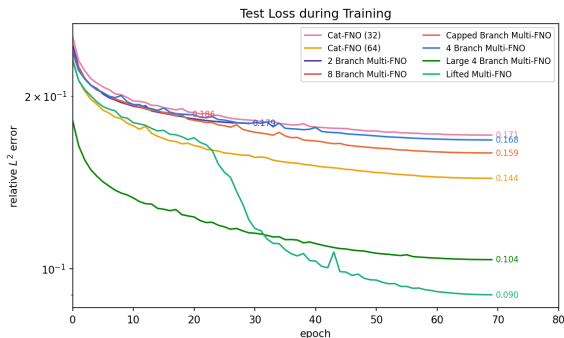


Figure: Test Error (unseen Re) during Training

Navier-Stokes Problem

On the airfoil problem, CAT-GEO-FNO and Lifted MFNO performed similarly at $\sim 3\%$.

For Navier-Stokes, Lifted-MFNO reaches 9.0% on unseen Re vs. 14.4% for CAT-FNO.

One reason could be because the range of the family is wider, preferring a multi-task operator over a single-task one.

However, note the standard Multi-FNO (which combines branches in output space instead of latent space) performs similarly compared to the CAT-FNO (16.8%).

Is the Multi-FNO mixing?

In our preliminary experiments, we found that a 4 Branch Multi-FNO was heavily favoring a single branch over the others during training/inference.

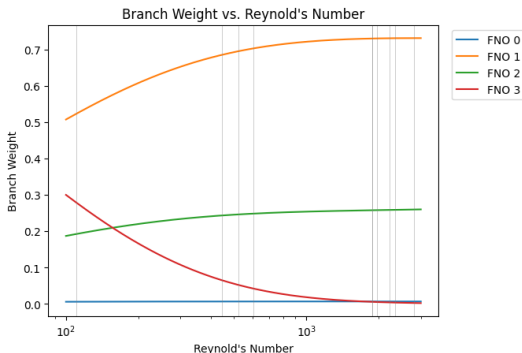


Figure: Branch weights vs Re (gray lines: unseen Re values)

Is the Multi-FNO mixing?

We tested if this behavior continues independent of the number of branches.

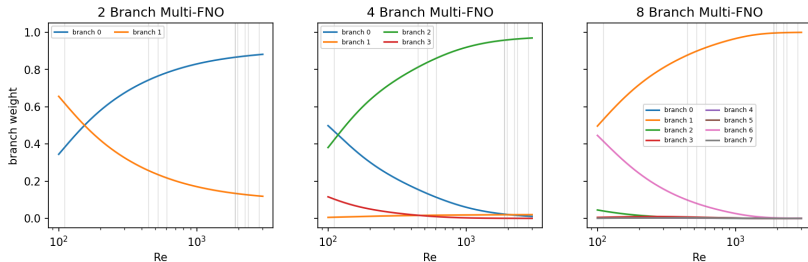


Figure: Branch weights across Re for Multi-FNOs with different numbers of branches

It turns out that the Multi-FNO always favors a single branch regardless of the number of branches it has.

Can we enforce mixing?

We tested penalizing branch weights above 50% during training.

This technique does force sharing and slightly improves the model (16.8% vs. 15.9%).

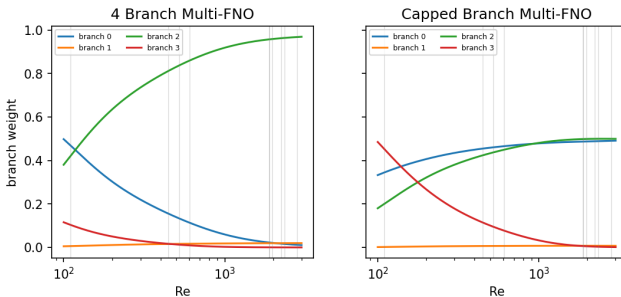


Figure: Branch Weights across Re for Capped-Branch Multi-FNOs

Future work: examine if the two major branches have similar outputs.

Model Evaluation

We evaluated the models using various metrics:

$$\text{Relative } L^2 \text{ error} := \frac{\|\hat{\omega} - \omega\|_2}{\|\omega\|_2} \quad (7)$$

$$\text{Relative } L^\infty \text{ error} := \frac{\max_x |\hat{\omega}(x) - \omega(x)|}{\max_x |\omega(x)|} \quad (8)$$

$$\begin{aligned} \text{Wrong rotation fraction} &:= \frac{1}{|M|} \sum_{x \in M} \mathbf{1}[\text{sign } \hat{\omega}(x) \neq \text{sign } \omega(x)], \\ M &:= \{x : |\omega(x)| > 0.2\sigma\} \end{aligned} \quad (9)$$

$$\begin{aligned} \text{Vortex overlap (IoU)} &:= \frac{|A \cap B|}{|A \cup B|}, \\ A &:= \{x : |\hat{\omega}(x)| > \sigma\}, \quad B := \{x : |\omega(x)| > \sigma\} \end{aligned} \quad (10)$$

Model Evaluation

The Lifted Multi-FNO performs better than the standard Multi-FNO over all evaluation metrics.

	Multi-FNO	Lifted MFNO
Relative L^2 error	16.8%	9.0%
Worst grid point (L^∞)	22.4%	13.6%
Wrong rotation direction	1.7%	0.27%
Vortex overlap (IoU)	0.824	0.902

90% of vortex cores land in the right place, and fewer than 0.3% of grid points spin the wrong way.

Error Distribution

Predicting trajectories is harder for higher Reynolds numbers.

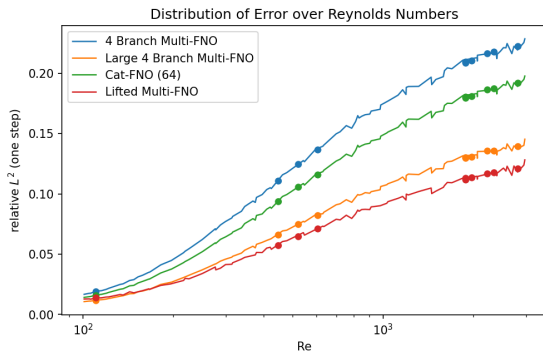


Figure: One-Step Error over entire Re range (dots: unseen Re)

The models are generalizing well to unseen Reynolds numbers within the distribution.

Error Distribution

Predicting trajectories is also harder for times later in the trajectory.

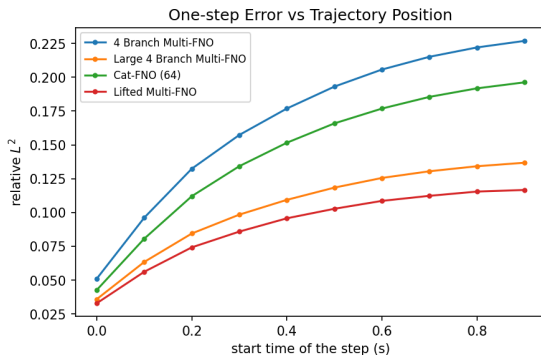


Figure: One-Step Error over entire Time range

This is likely due to the fact that later states are generally more turbulent and thus harder to predict.

Autoregressive Rollout

Similar to numerical methods, we can recursively apply our models to autoregressively predict trajectories.



Figure: Lifted MFNO rollout at unseen $Re = 110$

Autoregressive Rollout

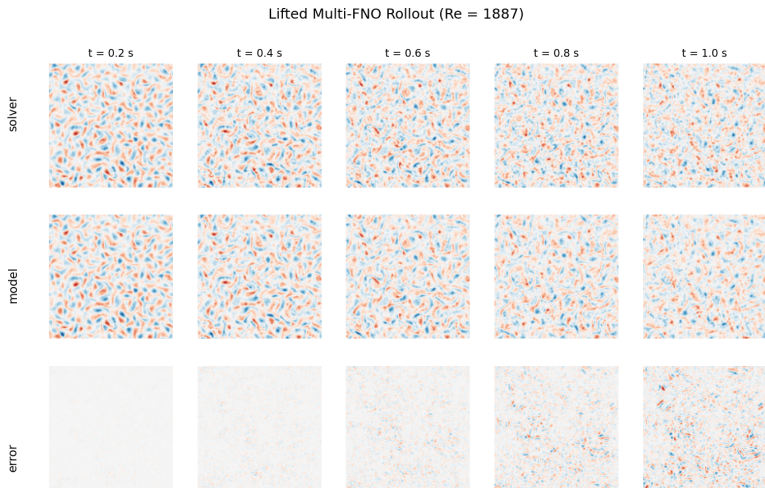


Figure: Lifted MFNO rollout at unseen $Re = 1886$

Autoregressive Rollout

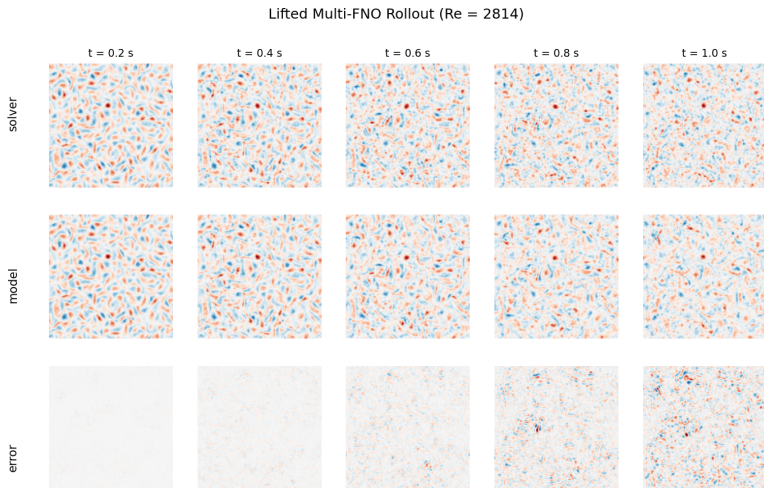


Figure: Lifted MFNO rollout at unseen $Re = 2814$

Autoregressive Rollout

Error accumulates linearly across time, even though one-step error is logarithmic across time.

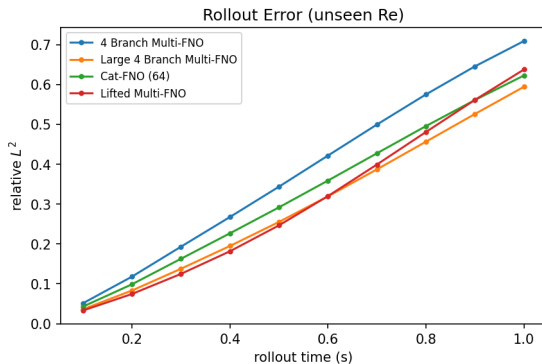


Figure: Autoregressive Rollout Error from $t = 0$ to $t = 1$ second

Testing Resolution Invariance

One strength of the FNO is it generalizes to unseen input resolutions.

We tested the models on 64×64 sized inputs.

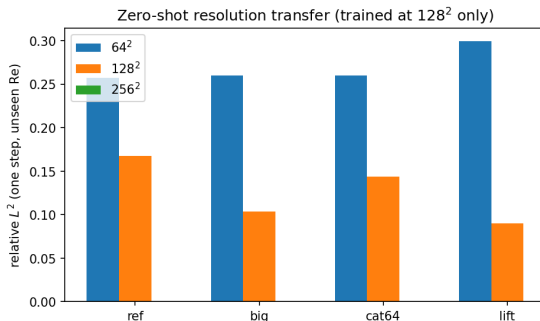


Figure: Error for 128×128 inputs and 64×64 inputs

The models do not generalize well to unseen input resolutions.

Conclusion

The Lifted-Multi-FNO architecture performs best when parameter counts are similar.

	Params	Unseen ICs	Unseen Re
CAT-FNO (width 32)	4.7M	13.4%	17.1%
CAT-FNO (width 64)	18.9M	11.2%	14.4%
Multi-FNO (4 branches)	18.9M	13.1%	16.8%
Multi-FNO (32 modes)	33.6M	8.1%	10.4%
Lifted MFNO	18.9M	7.1%	9.0%

However, the branched models generally prefer to weigh a single branch heavily, which can be mitigated using penalties.

High Reynolds numbers and states later in time are harder to predict.

The models do not generalize well to unseen input resolutions.

Future Work

Can we encourage better generalization by emphasizing training on higher Reynolds numbers?

Combining the results from our various experiments to create the best possible Lifted-Multi-FNO model.

Why do our models not generalize well towards unseen resolutions?

Scaling-laws for Multi-FNOs.

Thank You!

	Params	Unseen ICs	Unseen Re
CAT-FNO (width 32)	4.7M	13.4%	17.1%
CAT-FNO (width 64)	18.9M	11.2%	14.4%
Multi-FNO (4 branches)	18.9M	13.1%	16.8%
Multi-FNO (32 modes)	33.6M	8.1%	10.4%
Lifted MFNO	18.9M	7.1%	9.0%



Figure: Test Error (unseen Re) during Training